

Towards Distributed Validators on leanConsensus

DV Labs

February 2026

1 Introduction

Ethereum’s transition to proof-of-stake introduces a critical dependency on *validator signatures*: every attestation and block proposal must be signed by an authorised validator. The consensus layer currently relies on BLS12-381 for its native support for signatures aggregation. However, on the quantum horizon, BLS12-381 signatures will be broken by Shor’s algorithm running on a cryptographically-relevant quantum computer (CRQC). To address this issue, the Beacon chain is being redesigned as leanConsensus. Its core component is a hash-based zkVM aggregation system designed to address both the *post-quantum* and the *aggregation efficiency* requirements simultaneously. Its core contributions are:

- (i) A Cairo-inspired zkVM (leanVM) whose execution traces are proven by a WHIR-based SNARK.
- (ii) An aggregation circuit defined in a domain-specific language (DSL) that can aggregate arbitrarily many XMSS signatures in a single succinct proof via a hierarchical tree of sub-proofs.

This document provides an overview of leanConsensus, with particular focus on how it can be extended to support threshold signatures, and what open engineering challenges remain for distributed validator (DV) solutions.

2 Architecture of leanConsensus

2.1 Cryptographic Primitives

Base field. All arithmetic is performed over the *KoalaBear* prime field $\mathbb{F}_p$ with $p = 2^{31} - 2^{24} + 1$ (a 31-bit prime with 2-adicity 24). The extension used for polynomial commitments is $\mathbb{F}_{p^5}$, a degree-5 extension, giving a security level of 123 bits under the Johnson bound and 128 bits conjectured under the capacity bound. Every field element is 31 bits; an 8-element “digest” (the unit of hash output used throughout) is $31 \times 8 = 248$ bits.

Hash function. Poseidon2 with a 16-element ($16 \times 31 = 496$ -bit) state is used everywhere using the compression mode as it builds all Merkle trees in the system. Poseidon2 is algebraically friendly so that it integrates naturally into the AIR/Logup constraint system.

WOTS⁺ one-time signatures. WOTS⁺ is the leaf-level signing primitive. A WOTS⁺ private key $\mathbf{sk}_i = (s_1, \dots, s_V) \in \mathbb{F}_p^{V \cdot 8}$ generates a public key by iterating Poseidon2 along each chain. To sign, the hash encoding of the message is decomposed into per-chain lengths $e_1, \dots, e_V$ with $\sum e_i = T$, and each chain tip is revealed at position e_i from the start.

XMSS signatures. An XMSS key pair covers a lifetime of 2^{2^h} Ethereum *slots* where h is fixed at key generation time. The public key is a single digest $r_{\text{root}} \in \mathbb{F}_p^8$ representing the root of a Merkle tree of depth h over all WOTS⁺ leaf hashes. A signature at slot s consists of:

- The WOTS⁺ randomness (7 field elements).
- V chain tips ($V \times 8$ field elements).
- A Merkle authentication path of depth n ($n \times 8$ field elements).

2.2 The leanVM Execution Model

leanVM is a register-free, memory-based virtual machine inspired by Cairo [GPR21]. Its memory model is *write-once*: each address can be written at most once; a second write to the same address triggers a constraint violation. This property is central to correctness proofs. The instruction set includes:

- **Arithmetic:** ADD, MUL over $\mathbb{F}_p$ and $\mathbb{F}_{p^5}$.
- **Memory:** load from and write to the write-once memory.
- **Control flow:** unconditional and conditional jumps; the program counter advances monotonically, enabling efficient AIR constraints.
- **Precompiles:** poseidon16 and dot-product accelerators.

The execution trace is a table with three sub-tables: the *Execution table* (instruction columns + runtime witness columns), the *DotProduct table*, and the *Poseidon16 table*. The three tables are connected via the *Logup bus*: each precompile call inserts a (fingerprinted) row into the corresponding table, and the Logup argument [Hab22] proves that the multisets match.

2.3 Proving Pipeline

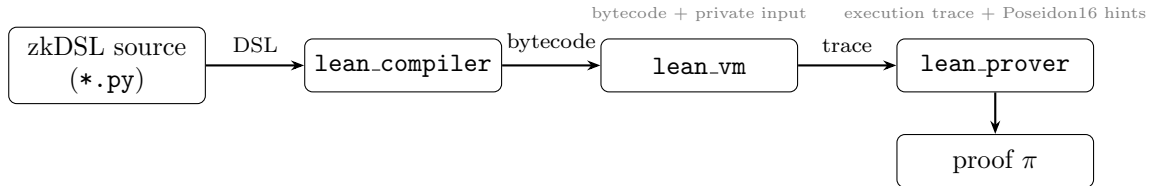


Figure 1: Proving pipeline

SuperSpartan. The main proof system is a variant of Spartan [Set20] operating over extension field multilinear polynomials. The execution table’s correctness is expressed as a sum over the Boolean hypercube of a degree-3 polynomial; a single sumcheck invocation reduces it to a point evaluation.

WHIR. Polynomial commitment proofs are generated with WHIR [ACFY25], a Reed–Solomon proximity protocol with:

- Initial folding factor 7, subsequent factor 5.
- 18 grinding bits (proof-of-work).
- Code rate $\rho \in \{1/2, 1/4, 1/8, 1/16\}$ selectable per node.

Logup. The lookup argument connecting the three tables follows the *logarithmic derivative* technique [Hab22]. The bus value is a linear combination of the table row, fingerprinted with Fiat-Shamir challenges.

2.4 Aggregation Tree Topology

A single `leanConsensus` node aggregates at most one “level” of the tree. Each node’s circuit (`main.py`) handles two source types:

1. **Raw XMSS** — individual validators signing directly.
2. **Recursive children** — previously-proven aggregation nodes whose proofs are verified inside this node’s circuit.

The public state of a node is an `AggregatedXMSS` structure:

- A list of public keys ($8 \times n$ field elements).
- A proof π (a few hundred KiB, constant regardless of n).
- A *bytecode claim* (an evaluation of the bytecode multilinear at a random point), used for recursive verification.

The global public key set is a deduplicated, sorted union of all raw and child public keys. Its *Poseidon-slice-hash* (i.e., the running hash obtained by sequentially hashing consecutive pairs of pubkey digests) is committed in the proof’s public input.

Bytecode claim reduction. When a node has n_r recursive children, it must prove that each child was proved with the same bytecode. The $2n_r$ bytecode claims (one per child, times two for the two halves of the recursion check) are collapsed into a single evaluation via a random linear combination and a sumcheck over the bytecode multilinear polynomial. The resulting evaluation point is propagated to the parent node.

3 Threshold Hash-Based Signatures

3.1 Motivation: Distributed Validators

In Ethereum’s Distributed Validator (DV) model [Fou23], a single logical validator is operated by a cluster of n independent nodes. This architecture merges threshold cryptography with Byzantine Fault Tolerant (BFT) consensus to eliminate single points of failure, optimize uptime, and prevent equivocation.

DKG and threshold signatures. Rather than relying on a trusted dealer to generate and distribute the private signing key, which would represent a central point of failure, the cluster performs a Distributed Key Generation (DKG) protocol. This allows the n operators to collectively compute a public verification key while each operator strictly learns only their secret share. In the pre-quantum BLS-based DV setting, the private signing key is split amongst multiple parties using a linear secret sharing scheme. To produce a threshold BLS12-381 signature, a signer signs the agreed-upon data with its key share. Because of the homomorphic properties unique to pairing-based schemes, k partial BLS signatures can be linearly combined into a single valid BLS signature that is indistinguishable from a standard, non-threshold signature.

Intra-cluster consensus and BFT. While threshold cryptography dictates *how* a signature is constructed, it does not dictate *what* is signed. Before any partial signatures are generated, the n nodes must run a BFT consensus protocol (e.g., an Istanbul BFT variant) to deterministically agree on the exact Beacon Chain duty (block or attestation data) they will execute. By enforcing consensus prior to signing, the DV guarantees that no honest operator will sign conflicting data, effectively eliminating the risk of accidental slashing.

System guarantees and threshold parameters. To tolerate malicious (Byzantine) actors within the cluster, the system parameters are typically bound by the BFT threshold $n \geq 3f + 1$, where f is the maximum number of Byzantine nodes. Consequently, the cryptographic signing threshold k is usually set to $k \geq 2f + 1$ (often represented as $k = \lceil \frac{2n}{3} \rceil$).

Impactical indistinguishable threshold hash-based signatures. Because the BLS homomorphic property abovementioned is unique to pairing-based schemes and does not transfer to hash-based signatures, under a post-quantum signature scheme like XMSS, threshold sharing via polynomial interpolation is not natively supported anymore. Therefore, supporting threshold signatures requires ad-hoc mechanisms. In order to be transparent at the protocol level, a key requirement for threshold signatures is to be indistinguishable from non-threshold ones to be compliant with the network specification. Amongst the different possible solutions, leveraging multi-party computation (MPC) has been shown to fall short in terms of performance [?] due to high number of hash calls to be computed in a distributed manner. A method based on Boolean sharing has also been proposed by Kelsey, Lang and Lucks in [KLL25] but it requires a trusted party and a large common reference string (around 1 GiB per key). Because both options are not satisfactory either in terms of performance or memory consumption, we hereafter consider other approaches that affect the signature format (i.e. threshold signatures are distinguishable from non-threshold ones).

3.2 Proof-Based Threshold Signatures

Leveraging proof systems to construct threshold hash-based signatures has already been explored by Khaburzaniya, Chalkias, Lewi and Malvai [KCLM22]. In this approach, an aggregator combines individual signatures over the same message into a zero-knowledge proof that serves as the actual threshold signature. More precisely, given a k -out-of- n setting, the aggregator can generate a proof attesting that it verified k distinct signatures over the same message and that signers are part of the quorum. A valuable feature of this approach is its non-interactiveness: the aggregator only needs to collect individual signatures in order to compute the proof, without any additional communication. On the other hand, it inflates the signature size by over an order of magnitude, making the resulting threshold signatures incompatible with standard signature formats. Furthermore, proof-based threshold signatures are ill-suited for the aggregation mechanisms central to leanVM. Integrating them introduces a new recursion layer for every threshold signature processed. Because each recursion layer takes an `AggregatedXMSS` alongside raw XMSS signatures as input, verifying the `AggregatedXMSS` within the circuit quickly becomes a severe performance bottleneck.

3.3 Dummy Threshold Signatures

Another approach would be to use *dummy* threshold signatures that simply consist of at least k individual signatures from authorized signers within a *threshold group* as defined below.

Definition 3.1 (Threshold Group). *A threshold group $G = (k, n, \mathcal{T})$ consists of:*

- k (threshold): *the minimum number of signers required.*
- n (group size): *the total number of members.*
- $\mathcal{T}$: *a Merkle tree of depth $d = \lceil \log_2 n \rceil$, whose 2^d leaves are the n individual public keys $r_1, \dots, r_n$ padded with zero digests to the nearest power of two.*

The joint public key of G is

$$\text{joint_key}(G) = \text{Poseidon}_{216}(r_G, [k_{\min}, 0, \dots, 0]) \in \mathbb{F}_p^8,$$

where $r_G = \mathcal{T}.\text{Root}$. The joint key cryptographically commits to both the membership set (via r_G) and the minimum signing threshold $k_{\min}$, so that neither can be forged independently once the key is registered.

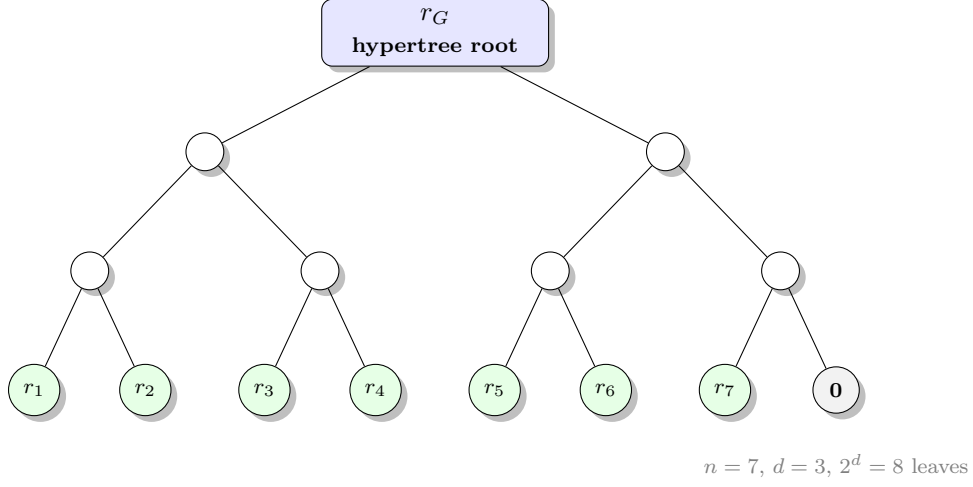


Figure 2: Hypertree for a k -out-of-7 threshold group ($n = 7$, depth $d = 3$). Seven XMSS public keys $r_1, \dots, r_7$ are placed as leaves; the eighth leaf is zero-padded. Each internal node is a Poseidon₂₁₆ compression of its two children. The joint public key registered on-chain is Poseidon₂₁₆($r_G, [k_{\min}, 0, \dots, 0]$), binding both the membership tree and the threshold.

A threshold signature then consists of a set of signer indices, individual signatures and membership proofs as defined hereafter.

Definition 3.2 (Threshold Signature). *Given a group $G = (k, n, \mathcal{T})$ and a message m , a threshold signature is a tuple $\Sigma = (\mathbf{i}, \sigma, \pi)$ where:*

- $\mathbf{i} = (i_1, \dots, i_k)$ is a set of k distinct signer indices, $i_j \in \{1, \dots, n\}$.
- $\sigma = (\sigma_1, \dots, \sigma_k)$ are the k individual XMSS signatures, σ_j being the signature of signer i_j on m .
- $\pi = (\pi_1, \dots, \pi_k)$ are the corresponding Merkle authentication paths, where π_j contains the d sibling hashes required to reconstruct the root r_G from the leaf r_{i_j} .

Threshold signature verification consists of three independent checks per signer:

1. **XMSS validity:** $\text{XMSS.Verify}(r_{i_j}, m, \sigma_j) = \top$.
2. **Hypertree membership:** the Merkle path π_j successfully authenticates the leaf r_{i_j} against the known joint public key r_G .
3. **Distinctness:** no index i_j appears more than once in $\mathbf{i}$.

The combined verification cost is dominated by the k XMSS verifications, each requiring $O(V \cdot L + \log_{\text{lifetime}})$ Poseidon2 calls (WOTS chains + XMSS Merkle path), plus $k \times d$ additional Poseidon2 calls for the hypertree paths.

4 Integration of Threshold Signatures into leanVM

4.1 Overview

The most straightforward way to extend leanVM in order to support dummy threshold signatures is to perform the verification of all k signers in a threshold group *inside the same aggregation circuit* as the raw XMSS and recursive proof checks. It implies supporting threshold signatures as a new source type in `main.py`, processed between raw XMSS and recursive sources. Unlike proof-based threshold signatures, the dummy threshold verification logic (XMSS verification of each signer plus $d \leq 3$ additional Poseidon2 hashes per signer for the hypertree path) is added as extra rows in the execution trace, not as an additional proof system invocation. The proving cost of extra trace rows is sublinear (only log factors enter through WHIR), so a threshold group with $k = 7$ signers adds roughly the same overhead as 7 extra raw XMSS signatures. Beyond the raw cost argument, it has the advantage to keep a single bytecode program and an homogeneous recursion interface as no new proof type is introduced. While Definition 3.1 is generic regardless of the n and d parameter, our leanVM implementation hard-asserts $n \leq 8$ (equivalently $d \leq 3$), as distributed validators rarely exceed 8 participants. Indeed most configurations use either $(k, n) = (3, 4)$ or $(5, 7)$.

4.2 In-Circuit Threshold Verification

The in-circuit implementation lives in `threshold_aggregate.py` and is compiled into the same bytecode as the rest of `main.py`.

4.2.1 Private Input Layout

Each threshold source block in private memory has the layout:

$$\underbrace{k}_{\text{signers}} \quad \underbrace{k_{\min}}_{\text{threshold}} \quad \underbrace{d}_{\text{depth}} \quad \underbrace{j}_{\text{pubkey idx}} \quad \underbrace{r_G}_{\text{root hint}} \quad \underbrace{\ell_1, \dots, \ell_k}_{\text{leaf indices}} \quad \underbrace{r_{\ell_1}, \dots, r_{\ell_k}}_{\text{signer roots (hints)}} \quad \underbrace{\sigma_1, \dots, \sigma_k}_{\text{XMSS sigs}} \quad \underbrace{\pi_1, \dots, \pi_k}_{\text{hypertree proofs}}$$

Here k is the number of signers who actually participated in this instance (may exceed $k_{\min}$), $k_{\min}$ is the registered minimum threshold, $r_G \in \mathbb{F}_p^8$ is a private hint of the hypertree root (verified in-circuit against the registered `joint_key`), each signer root $r_{\ell_i} \in \mathbb{F}_p^8$ is a private hint recomputed and checked during XMSS verification, and each hypertree proof π_i consists of d sibling digests. The global private input header was extended from the pre-threshold format:

Layout	Description
<code>[n recursions]</code>	Number of recursive child proofs
<code>[n threshold]</code>	Number of threshold source blocks
<code>[n dup]</code>	Number of duplicate public keys
<code>[ptr pubkeys]</code>	Pointer to global public key table
<code>[ptr_src_0, ptr_thresh_0..t, ptr_rec_0..n, ptr_bytecode_sumcheck]</code>	Ordered source pointers

4.2.2 Verification Algorithm

For each threshold source block (indexed $t = 0, \dots, n_{\text{threshold}} - 1$), the circuit executes Algorithm 1.

Algorithm 1 `threshold_verify_group` (in-circuit, zkDSL)

Require: k : actual signer count; $k_{\min}$: minimum threshold; d : depth; j : global pubkey index; r_G : hypertree root; $\ell_1, \dots, \ell_k$: leaf indices; r_{ℓ_i} : signer root hints; σ_i : XMSS signatures; π_i : hypertree proofs

```
1: assert  $0 < k_{\min} \leq k \leq \text{MAX\_THRESHOLD\_SIGNERS}$  ▷ threshold enforcement
2: assert  $0 < d$ 
3:  $\hat{j}k \leftarrow \text{Poseidon2}_{16}(r_G, [k_{\min}, 0, \dots, 0])$ 
4: assert  $\hat{j}k = \text{all\_pubkeys}[j]$  element-wise ▷ verify  $r_G$  and  $k_{\min}$  against registered joint key
5:  $\text{seen} \leftarrow \text{Array}(\text{MAX\_THRESHOLD\_SIGNERS})$  ▷ write-once bitmap for distinctness
6: for  $i = 0 \rightarrow k - 1$  do
7:   assert  $\ell_i < 2^d$  ▷ leaf index in bounds
8:    $\text{seen}[\ell_i] \leftarrow i + 1$  ▷ write-once: duplicate  $\ell_i$  triggers MemoryAlreadySet
9:    $\text{XMSS.InCircuitVerify}(r_{\ell_i}, m, \sigma_i, \text{slot})$  ▷ writes  $r_{\ell_i}$  to its hint address; memory-write assertion
10:   $\hat{r}_G \leftarrow \text{HypertreeMerkleVerify}(r_{\ell_i}, \pi_i, \ell_i, d)$ 
11:  assert  $\hat{r}_G = r_G$  element-wise ▷ signer is a member of the group
12: end for
13: return  $j$  ▷ global pubkey index used for partition counter
```

4.2.3 Distinctness via Write-Once Memory

The bitmap array `seen[ $\text{MAX\_THRESHOLD\_SIGNERS}$ ]` uses the write-once memory property of `leanVM` to enforce signer index uniqueness without a sorting circuit or a hash set. On the first write, $\text{seen}[\ell_i] \leftarrow i + 1$ succeeds. On a second write with any other value (since i is strictly increasing), the memory constraint

$$(\text{previous value}) = (\text{new value})$$

is violated, causing an unsatisfiable AIR constraint. This is provably sound under the Logup bus argument: the memory table enforces that every read is consistent with the unique write that created the entry.

4.2.4 Hypertree Merkle Verification (Depth Dispatch)

Because the maximum depth is $d \leq 3$ and is known at compile time from the `MAX_HYPERTREE_DEPTH` placeholder, the circuit dispatches statically on d rather than using a variable-depth loop:

Listing 1: Depth dispatch in `threshold_aggregate.py`

```
1 @inline
2 def hypertree_merkle_verify(leaf_digest, proof_path, leaf_idx, depth, out_root):
3     match depth:
4         case 1: _hypertree_verify_d1(leaf_digest, proof_path, leaf_idx, out_root)
5         case 2: _hypertree_verify_d2(leaf_digest, proof_path, leaf_idx, out_root)
6         case 3: _hypertree_verify_d3(leaf_digest, proof_path, leaf_idx, out_root)
7     return
```

Each `_hypertree_verify_d` function is a fixed chain of `Poseidon216` calls with a `match` on the bit decomposition of `leaf_idx`. The bit decomposition is *big-endian*: bit index $\text{depth} - 1 - j$ controls the direction (left/right) at Merkle level j (counted from the leaves). Left child means $b = 0$: `Poseidon2(current, sibling)`; right child means $b = 1$: `Poseidon2(sibling, current)`. See Listing 2 for an example with depth 3.

Listing 2: Depth-3 Merkle verification

```
1 def _hypertree_verify_d3(leaf_digest, proof, leaf_idx, out_root):
2     bits = checked_decompose_bits_small_value_const(leaf_idx, 3) # [MSB, mid, LSB]
3     temp0 = Array(DIGEST_LEN)
4     temp1 = Array(DIGEST_LEN)
5     match bits[2]: # level 0 (leaves): LSB
6         case 0: poseidon16(leaf_digest, proof, temp0)
7         case 1: poseidon16(proof, leaf_digest, temp0)
```

```

8      match bits[1]: # level 1
9          case 0: poseidon16(temp0, proof + DIGEST_LEN, temp1)
10         case 1: poseidon16(proof + DIGEST_LEN, temp0, temp1)
11      match bits[0]: # level 2 (root's children): MSB
12         case 0: poseidon16(temp1, proof + 2*DIGEST_LEN, out_root)
13         case 1: poseidon16(proof + 2*DIGEST_LEN, temp1, out_root)
14      return

```

4.2.5 Root Assertion Pattern

A subtle correctness point: the signer root r_{ℓ_i} passed into `xmss.verify` is a *hint* in the private input at a known address. The XMSS circuit writes the re-derived root to that address via the memory-write assertion. The hypertree circuit then reads that same address to begin the Merkle path computation. The hypertree root $\hat{r}_G$ is computed into a *fresh* buffer (not into the `all_pubkeys` region), because that region may already have been written. An explicit element-wise assertion

$$\hat{r}_G[j] = r_G[j], \quad j = 0, \dots, 7$$

is emitted rather than relying on the write-once memory assertion pattern. This design choice makes correctness independent of the memory state of `all_pubkeys`.

4.3 Poseidon Precomputation

The leanProver requires all Poseidon2₁₆ input–output pairs to be provided before trace generation (the Poseidon2₁₆ table is populated from hints, not re-evaluated in the execution trace). For a threshold group, the complete trace comprises:

- One Poseidon2₁₆ call for the joint-key commitment: Poseidon2₁₆($r_G, [k_{\min}, 0, \dots, 0]$).
- For each of the k signers: all Poseidon2 calls from WOTS chain evaluation and XMSS Merkle path.
- For each of the k signers: $d \leq 3$ additional Poseidon2 calls for the hypertree authentication path.

The function `threshold_verify_with_poseidon_trace` in `crates/xmss/src/hypertree.rs` performs native verification while collecting the full trace:

Listing 3: Trace collection for joint key and hypertree path (Rust, `hypertree.rs`)

```

1 // Joint key commitment: poseidon16(r_G, [k_min, 0, ..., 0])
2 let k_min_digest: Digest = std::array::from_fn(|i| {
3     if i == 0 { F::from_usize(group.k) } else { F::default() }
4 });
5 let _ = poseidon16_compress_with_trace(
6     &group.root(), &k_min_digest, &mut poseidon_trace);
7
8 // Hypertree authentication path for each signer
9 for sibling in &tsig.merkle_proofs[i] {
10     let is_left = (idx & 1) == 0;
11     current = if is_left {
12         poseidon16_compress_with_trace(&current, sibling, &mut poseidon_trace)
13     } else {
14         poseidon16_compress_with_trace(sibling, &current, &mut poseidon_trace)
15     };
16     idx >>= 1;
17 }

```


5 Key generation for Distributed Validators

In the pre-quantum DV model, an operator cluster registers a single BLS key on the Ethereum beacon chain. Under leanConsensus, the registration changes: the cluster registers a single 248-bit *joint public key*

$$\text{joint_key} = \text{Poseidon2}_{16}(r_G, [k_{\min}, 0, \dots, 0])$$

that simultaneously commits to the full membership set (via the hypertree root r_G) and the minimum signing threshold $k_{\min}$. The hypertree is constructed off-chain by any cluster member (or a trusted coordinator) as:

1. Each node independently generates an XMSS key pair (sk_i, pk_i) with $pk_i = r_i \in \mathbb{F}_p^8$.
2. All n roots $r_1, \dots, r_n$ are shared to all nodes that are part of the DV cluster.
3. Any member computes $\mathcal{T}$, derives $r_G = \mathcal{T}.\text{Root}$, and computes joint_key .
4. joint_key is submitted for on-chain registration.

No trusted dealer is required, and no secret is shared between operators: each holds only their own XMSS key.

6 Open Challenges

6.1 Cluster Mutability and Key Rotation

The problem. The joint public key $\text{joint_key} = \text{Poseidon2}_{16}(r_G, [k_{\min}, 0, \dots, 0])$ is a static commitment to a fixed membership set and a fixed threshold. Once registered on-chain, it cannot be updated without a new on-chain transaction. Real DV clusters need to handle:

- *Member churn*: an operator leaves and is replaced.
- *Key rotation*: an operator suspects key compromise and generates a fresh XMSS key pair.
- *Threshold adjustment*: the cluster policy changes (e.g., from 3-of-5 to 4-of-5 after detecting an unreliable member).

Current limitation. The current implementation has no on-chain mechanism for updating the joint key. A cluster that needs to change membership or threshold must:

1. Generate a new hypertree $\mathcal{T}'$ with updated members and compute the new joint key.
2. Submit an exit transaction for the old validator (identified by its joint_key).
3. Register a new validator with the new joint key.

This process requires waiting for the exit queue and incurs a new deposit.

6.2 Aggregation Throughput and the Stake-Alignment Problem

Circuit cost of threshold groups. In leanConsensus, the aggregation proof has a capacity determined by the proving infrastructure. Each raw XMSS signature consumes one unit of circuit capacity (one in-circuit XMSS verification). A k -of- n threshold group consumes k units — one per participating signer — while covering a single logical validator slot. If k "small" validators (each holding the minimum stake S) pool into a DV cluster registered as a single validator with stake S , the group consumes k circuit slots for S worth of stake. The effective cost ratio is k circuit slots per S , versus 1 circuit slot per S for a raw validator. Left unchecked, this would allow validators to consume $k \times$ more proof resources for the same stake contribution, creating a free-rider problem: the aggregation infrastructure is degraded for all participants.

Vitalik’s stake-alignment rule. To prevent this, Buterin [But25] proposes requiring a k -of- n DV cluster to hold $n \times S$ in total stake, with the group counted as n virtual validators each holding S . This aligns the stake contribution with the group size rather than the signer count. Under this rule, the proof-slot efficiency (staked ETH per circuit slot consumed per epoch) becomes

$$\eta_{\text{DV}} = \frac{n \times S}{k} \geq S.$$

DVs are therefore *at least as circuit-efficient as raw validators per unit of stake*. In fact, because the group has n validator-slots but only k circuit slots, it is strictly more efficient whenever $k < n$. Concretely, replacing n individual validators by one k -of- n DV (with $n \times S$ stake) reduces the epoch-wide circuit load by $n - k$ slots. For typical DV parameters, the net gain is 2 circuit slots per DV for $(k, n) = (5, 7)$ and 1 slot for $(k, n) = (3, 4)$. The $n \times S$ rule thus simultaneously eliminates the free-rider problem and improves overall aggregation throughput relative to a same-stake baseline of individual validators.

Exclusion of small stakers. The $n \times S$ requirement carries a substantial access barrier. With the current minimal staking balance being 32 ETH, meeting it demands locked capital of $n \times S$ leads to 128 ETH for a minimal 3-of-4 cluster. Individual small stakers, including participants in liquid staking protocols who contribute far less than 32 ETH apiece, cannot meet this threshold independently. A bit less restrictive rule would require only $k \times S$ in stake, since k is the direct driver of circuit cost.

References

- [ACFY25] Gal Arnon, Alessandro Chiesa, Giacomo Fenzi, and Eylon Yogev. Whir: Reed-solomon proximity testing with super-fast verification. In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025*, pages 214–243, Cham, 2025. Springer Nature Switzerland.
- [But25] Vitalik Buterin. Native DVT for Ethereum staking. <https://ethresear.ch/t/native-dvt-for-ethereum-staking/23894>, 2025. Ethereum Research forum post, accessed: 2025-02-27.
- [Fou23] Ethereum Foundation. Distributed validator technology. <https://ethereum.org/en/staking/dvt/>, 2023. Accessed: 2025-02-10.
- [GPR21] Lior Goldberg, Shahar Papini, and Michael Riabzev. Cairo – a Turing-complete STARK-friendly CPU architecture. Cryptology ePrint Archive, Paper 2021/1063, 2021.
- [Hab22] Ulrich Haböck. Multivariate lookups based on logarithmic derivatives. Cryptology ePrint Archive, Paper 2022/1530, 2022.
- [KCLM22] Irakliy Khaburzaniya, Konstantinos Chalkias, Kevin Lewi, and Harjasleen Malvai. Aggregating and Thresholdizing Hash-based Signatures using STARKs. In *Proceedings of the 2022 ACM on Asia Conference on Computer and Communications Security*, ASIA CCS ’22, page 393–407, New York, NY, USA, 2022. Association for Computing Machinery.
- [KLL25] John Kelsey, Nathalie Lang, and Stefan Lucks. Turning Hash-Based Signatures into Distributed Signatures and Threshold Signatures. *IACR Communications in Cryptology*, 2(2), 2025.

- [Set20] Srinath Setty. Spartan: Efficient and general-purpose zksnarks without trusted setup. In Daniele Micciancio and Thomas Ristenpart, editors, *Advances in Cryptology – CRYPTO 2020*, pages 704–737, Cham, 2020. Springer International Publishing.